Universidad Autónoma de Baja California



Diseño de la Arquitectura Integral del Sistema

SafeBand: pulseras de seguridad infantil

Proyecto de carrera

Fernando Onit Sibaja Gomez

Matrícula: 1279781

12/10/2026

Diagrama general de arquitectura

SafeBand se organiza en tres capas. En la escuela, los dispositivos de campo (pulseras, nodos receptores y terminales) generan los eventos y se conectan por WiFi a la red del plantel. En la nube, una sola API en ASP.NET Core recibe los eventos, aplica las reglas de negocio y los guarda en SQL Server. Los clientes (padres y personal) usan el mismo panel web, instalable como PWA, que muestra a cada quien solo lo que su rol permite.

La pulsera nunca se conecta a Internet: solo anuncia por BLE y se identifica por NFC. Esto mantiene su consumo bajo y concentra la conectividad en los nodos y las terminales, que tienen alimentación fija.

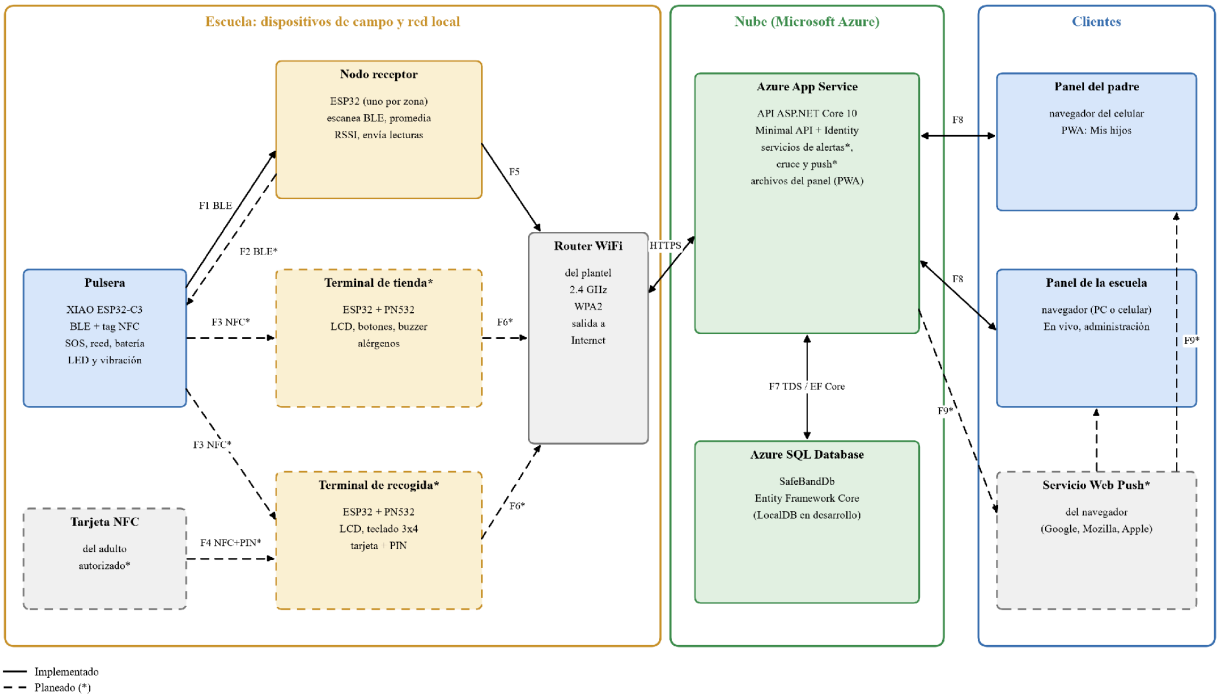


Figura 1. Diagrama general de arquitectura con los flujos de datos F1 a F9.

Diagrama de despliegue

La Figura 2 muestra en qué equipo corre cada artefacto. El firmware se compila en la PC de desarrollo con ESP-IDF y se graba por USB en cada placa. La API y los archivos del panel se publican juntos en un solo App Service de Azure, y la base de datos vive en Azure SQL. Mientras la publicación en Azure está pendiente (RNF-15), el mismo artefacto corre en la PC de desarrollo con SQL Server LocalDB y se expone en la red local para que los nodos le envíen datos.

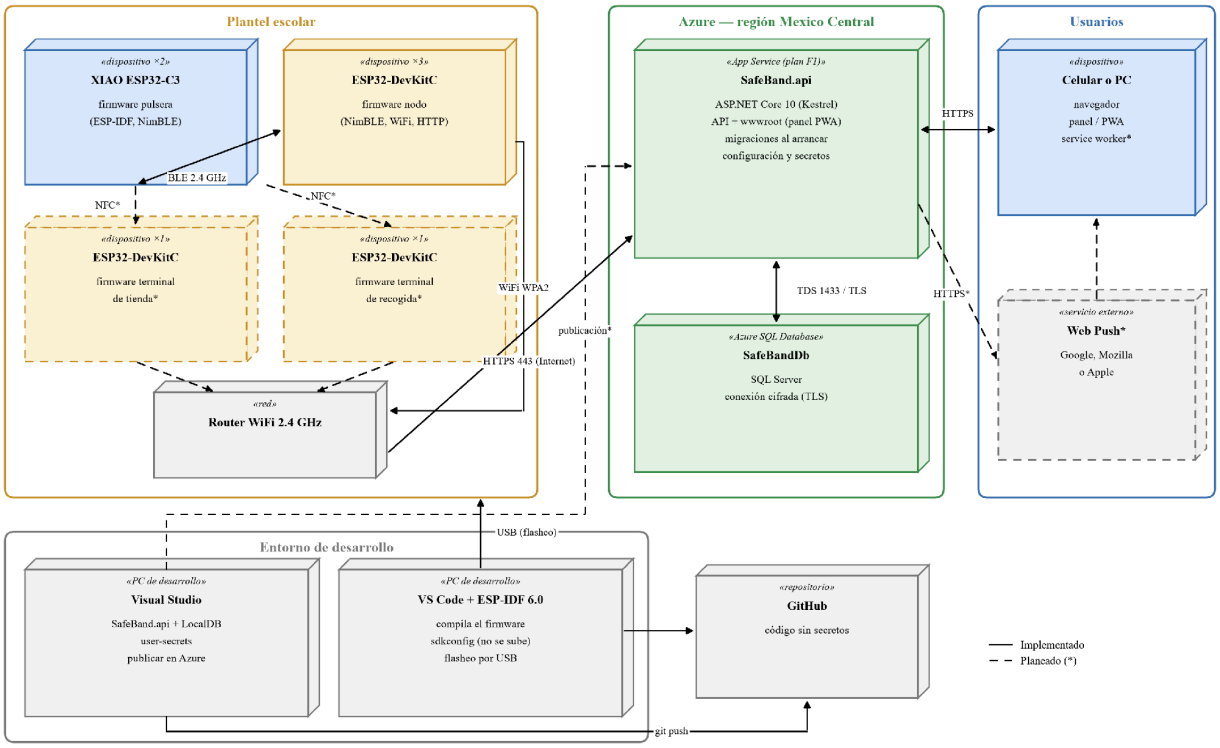


Figura 2. Diagrama de despliegue.

Diagrama de hardware

La Figura 3 muestra los cuatro tipos de dispositivo con sus periféricos y conexiones. Todos los microcontroladores trabajan con lógica de 3.3 V; los periféricos de 5 V (LCD y buzzer) se alimentan del pin de 5 V del USB y se controlan con un transistor o con el bus I2C ajustado a 3.3 V.

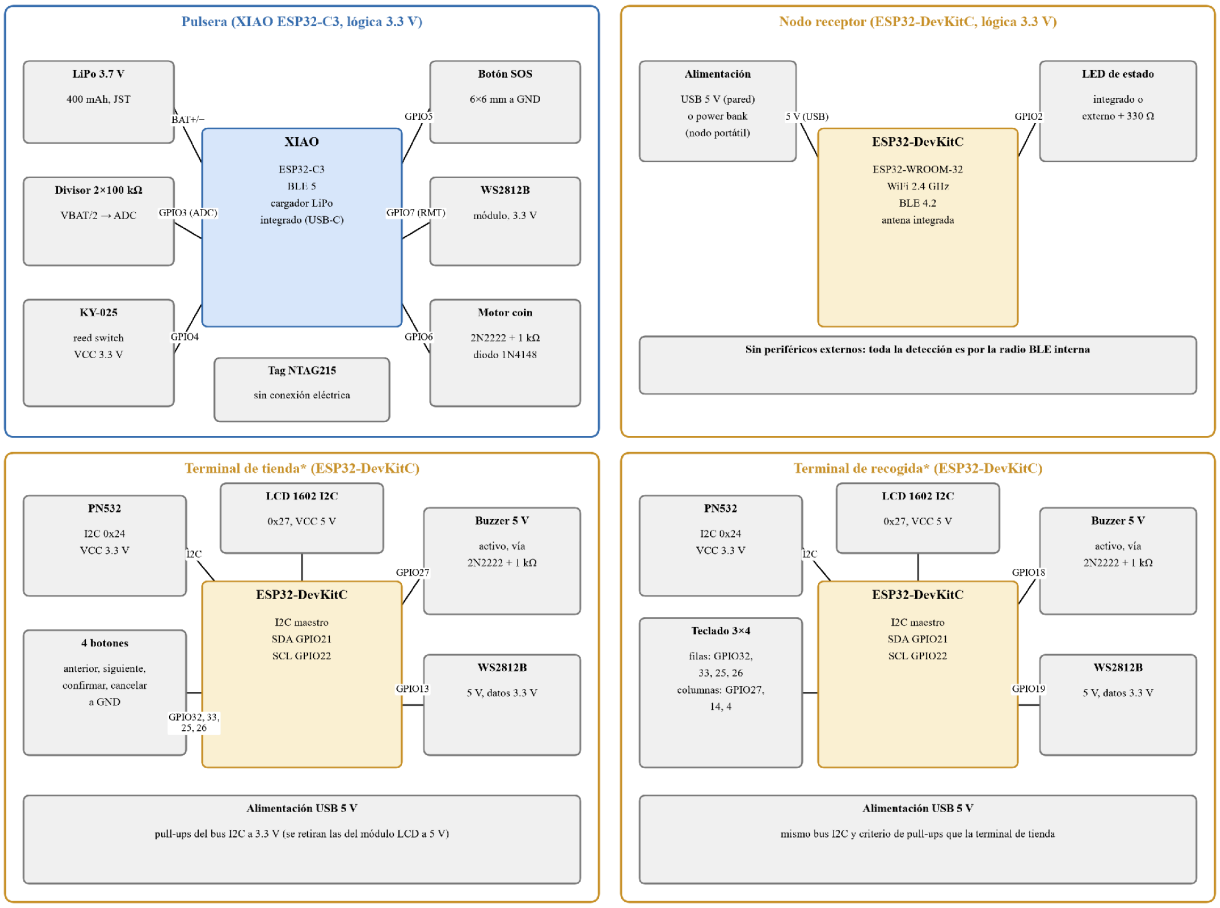


Figura 3. Diagrama de hardware de la pulsera, el nodo y las terminales (asignación preliminar).

Mientras se ensambla la pulsera definitiva, el prototipo usa un ESP32 genérico que simula las entradas: botón BOOT (GPIO0) como SOS, un puente a tierra en GPIO25 como «pulsera puesta», un puente en GPIO26 como «batería baja» y el LED integrado (GPIO2) como indicador. Este firmware ya implementa el formato de anuncio definitivo, por lo que el cambio a la XIAO solo modifica la asignación de pines.

Diagrama de módulos de software

La Figura 4 descompone el software en sus cuatro partes: firmware, backend, base de datos y frontend. Hoy el firmware de cada dispositivo está en un solo archivo main.c organizado por secciones; cada sección corresponde a un módulo de la figura y podrá separarse en un componente de ESP-IDF sin cambiar su interfaz.

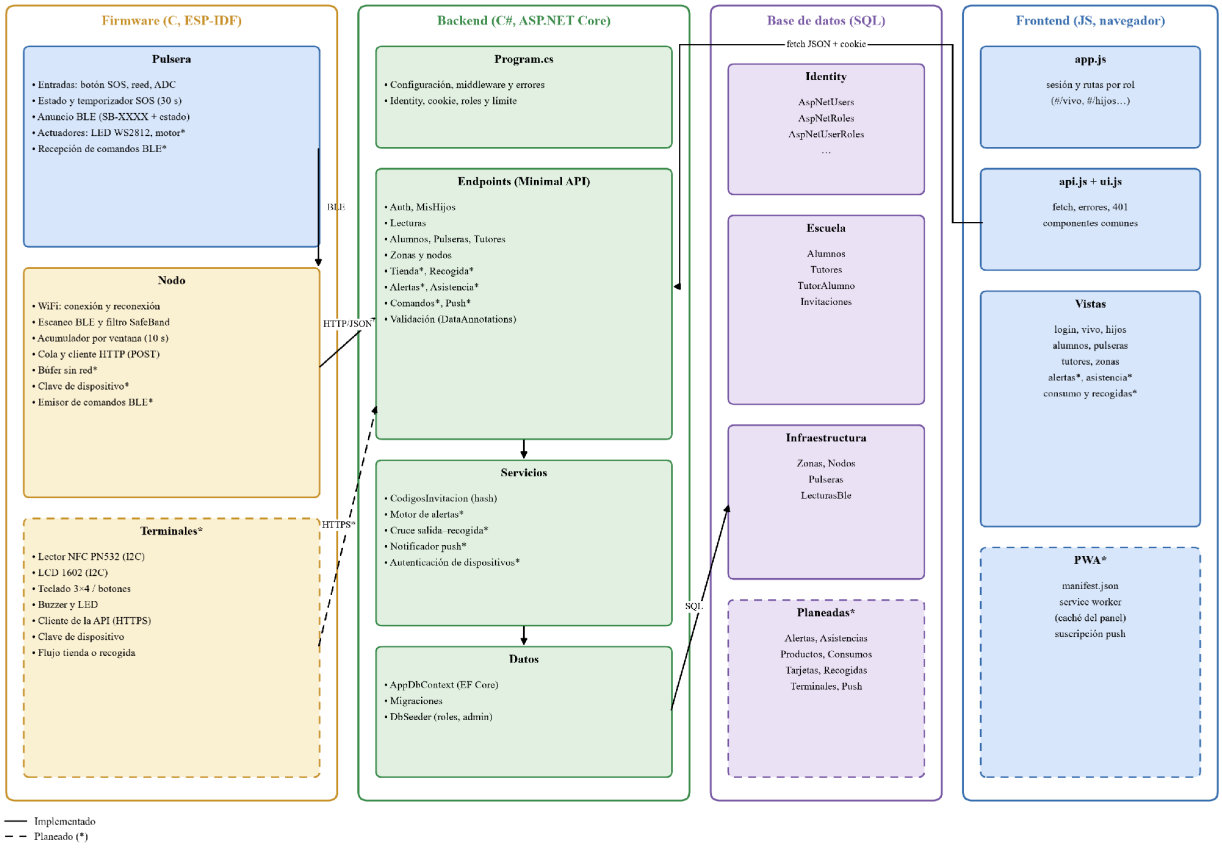


Figura 4. Diagrama de módulos de software.

Responsabilidad de cada módulo

Parte	Módulo	Responsabilidad	Estado
Firmware de la pulsera	Entradas	Leer cada 50 ms el botón SOS, el reed switch y el voltaje de batería, con anti-rebote y pulsación de 2 s para el SOS.	Implementado
	Estado y anuncio BLE	Armar el byte de estado y anunciar SB-XXXX cada 500 ms (100 ms durante el SOS de 30 s).	Implementado
	Actuadores	Indicar el estado con el LED WS2812 (y vibración*): SOS, pulsera retirada, batería baja, llamado y resultado de recogida.	Parcial (LED simple)
	Comandos BLE*	Escanear periódicamente los comandos firmados del nodo (simulacro, llamado y resultado de recogida) y aplicarlos.	Planeado
Firmware del nodo	WiFi	Conectarse a la red configurada, registrar el motivo de cada desconexión y reconectarse solo.	Implementado
	Escaneo y acumulador	Escanear BLE de forma pasiva, ignorar lo que no sea SafeBand y promediar el RSSI por pulsera en ventanas de 10 s.	Implementado

Parte	Módulo	Responsabilidad	Estado
	Envío HTTP	Enviar una lectura por pulsera al final de cada ventana y el SOS de inmediato (máximo uno cada 5 s por pulsera).	Implementado
	Búfer, clave y comandos*	Guardar lecturas sin red, autenticarse con su clave y anunciar los comandos que le entregue la API.	Planeado
Firmware de terminales*	NFC, LCD, teclado o botones	Leer el UID de tarjetas y pulseras, mostrar mensajes, capturar el PIN o el producto y accionar buzzer y LED.	Planeado
	Cliente de la API	Consultar y registrar en la API con su clave de dispositivo; sin conexión, no autorizar ni registrar nada.	Planeado
Backend	Program.cs y middleware	Configurar servicios, Identity, cookie, políticas por rol, límite de intentos, manejo global de errores y migraciones al arrancar.	Implementado
	Endpoints	Exponer la API REST: validar la entrada, aplicar permisos por rol y traducir a operaciones de datos.	Implementado (parte planeada)
	Servicios	Códigos de invitación; motor de alertas, cruce salida–recogida, notificador push y autenticación de dispositivos*.	Parcial
	Datos	AppDbContext con restricciones e índices, migraciones y sembrado inicial (roles y administrador).	Implementado
Base de datos	Esquema SQL Server	Guardar el historial con integridad referencial, bajas lógicas y fechas en UTC.	Implementado (tablas planeadas)
Frontend	app.js	Verificar la sesión, construir el menú según el rol y cambiar de vista con la ruta (#/…).	Implementado
	api.js y ui.js	Centralizar las peticiones, los mensajes de error y la sesión vencida; componentes visuales comunes.	Implementado
	Vistas	Pantallas de inicio de sesión, En vivo, Mis hijos y administración; alertas, asistencia, consumo y recogidas*.	Implementado (parte planeada)
	PWA*	Manifest, service worker con caché del panel y suscripción a notificaciones push.	Planeado

Flujo de datos entre los componentes

Los flujos de la tabla corresponden a las flechas numeradas de la Figura 1. El flujo principal, ya implementado de extremo a extremo, es F1 → F5 → F7 → F8: la pulsera anuncia, el nodo resume y envía, la API guarda y el panel consulta.

Flujo	Origen → destino	Dato	Protocolo y formato	Frecuencia o disparo	Estado
F1	Pulsera → nodo	Identificador y byte de estado	BLE, anuncio no conectable	Cada 500 ms; 100 ms en SOS	Implementado
F2	Nodo → pulsera	Comando firmado (simulacro, llamado, resultado)	BLE, anuncio del nodo	Mientras el comando esté vigente	Planeado
F3	Pulsera → terminal	UID del tag NTAG215	NFC ISO/IEC 14443A	Al acercar la pulsera	Planeado

Flujo	Origen → destino	Dato	Protocolo y formato	Frecuencia o disparo	Estado
F4	Tarjeta y adulto → terminal de recogida	UID de la tarjeta y PIN	NFC + teclado	En cada recogida	Planeado
F5	Nodo → API	Lectura (RSSI promedio y estado); consulta de comandos*	HTTP(S) + JSON	Cada 10 s por pulsera; SOS inmediato	Implementado
F6	Terminales → API	Verificación y registro de consumo y recogida	HTTPS + JSON	En cada operación	Planeado
F7	API ↔ base de datos	Entidades y consultas	TDS mediante EF Core	En cada petición	Implementado
F8	Panel ↔ API	Pantallas, datos y sesión	HTTPS + JSON, cookie	En vivo cada 5 s; Mis hijos cada 10 s	Implementado
F9	API → servicio push → navegador	Notificación de alerta o recogida	Web Push (VAPID)	Al crearse la alerta o la recogida	Planeado

Tabla de entradas y salidas

Entradas y salidas físicas y lógicas de cada componente. Las señales digitales de los microcontroladores trabajan a 3.3 V.

Componente	E/S	Señal o dato	Tipo	Descripción
Pulsera	Entrada	Botón SOS	Digital, pull-up interno	Activo en bajo; cuenta como SOS si se mantiene 2 s.
	Entrada	Reed switch KY-025	Digital	Indica si el broche está cerrado (pulsera puesta).
	Entrada	Voltaje de batería	Analógica (ADC)	VBAT/2 por divisor; batería baja si VBAT < 3.3 V.
	Salida	LED WS2812B	Digital serie (RMT)	Color según estado: SOS, retirada, batería baja, llamado*, verde/rojo de recogida*.
	Salida	Motor de vibración*	Digital vía transistor	Aviso háptico de SOS, llamado y simulacro.
	Salida	Anuncio BLE	Radio	Identificador y byte de estado.
	Entrada	Comando BLE*	Radio	Simulacro, llamado y resultado de recogida.
Nodo	Entrada	Anuncios BLE	Radio	Anuncios de las pulseras con su RSSI.
	Salida	Lectura	HTTP + JSON	POST /api/lecturas por pulsera y ventana.
	Salida	Comando BLE*	Radio	Retransmite los comandos que entrega la API.
Terminal de tienda*	Entrada	UID de la pulsera	I2C (PN532)	Identifica al alumno.

Componente	E/S	Señal o dato	Tipo	Descripción
	Entrada	4 botones	Digital, pull-up interno	Elegir producto, confirmar o cancelar.
	Salida	LCD, buzzer y LED	I2C / digital	Producto, advertencia de alérgeno y confirmación.
Terminal de recogida*	Entrada	UID de tarjeta y de pulsera	I2C (PN532)	Identifica al adulto y al alumno.
	Entrada	Teclado 3×4	Matriz digital	Captura el PIN y la elección del alumno.
	Salida	LCD, buzzer y LED	I2C / digital	Instrucciones y resultado (autorizada o rechazada).
API	Entrada	Peticiones HTTP	JSON	Lecturas, operaciones de terminales y del panel.
	Salida	Respuestas y notificaciones	JSON / Web Push*	Datos para el panel, códigos de estado y avisos.
Panel web	Entrada	Acciones del usuario	Formularios, clics	Inicio de sesión, altas, consultas.
	Salida	Pantallas	HTML	Ubicación, estado, recorrido, alertas* e invitaciones.

Asignación preliminar de pines y niveles de voltaje

Criterios

- Todos los GPIO del ESP32 y del ESP32-C3 trabajan a 3.3 V y no toleran 5 V; ninguna señal de 5 V se conecta directo a un pin.
- Se evitan los pines de arranque (strapping): GPIO2, GPIO8 y GPIO9 en el ESP32-C3; GPIO0, GPIO2, GPIO5, GPIO12 y GPIO15 en el ESP32, así como GPIO6 a GPIO11, reservados para la memoria flash.
- Las entradas de botones y teclado usan pull-up interno; por eso no se usan GPIO34 a GPIO39, que no lo tienen.
- Las cargas que consumen más de 20 mA (motor y buzzer) se activan con un transistor 2N2222 con resistencia de base de 1 kΩ; el motor lleva diodo de protección.
- El bus I2C (PN532 y LCD) trabaja a 3.3 V: el PN532 se alimenta a 3.3 V y se retiran o se sustituyen las resistencias pull-up del módulo de la LCD, que se alimenta a 5 V.

Pulsera (Seeed XIAO ESP32-C3)

Señal	Pin	Voltaje	Notas
Batería (ADC)	D1 / GPIO3	0–2.1 V	Divisor de 2 × 100 kΩ desde VBAT (3.0–4.2 V); atenuación de 12 dB en el ADC.
Reed switch	D2 / GPIO4	3.3 V	Salida digital del KY-025, alimentado a 3.3 V.
Botón SOS	D3 / GPIO5	3.3 V	A GND, con pull-up interno.
Motor de vibración	D4 / GPIO6	3.3 V lógica; motor a VBAT	Base del 2N2222 por 1 kΩ; diodo 1N4148 en paralelo con el motor.

Señal	Pin	Voltaje	Notas
LED WS2812B	D5 / GPIO7	3.3 V	Datos por el periférico RMT; módulo alimentado a 3.3 V.
Batería LiPo	BAT+ / BAT−	3.7 V nominal	400 mAh; se carga por el USB-C de la XIAO.

Nodo receptor (ESP32-DevKitC)

Señal	Pin	Voltaje	Notas
LED de estado	GPIO2	3.3 V	Opcional: LED integrado de la placa o externo con 330 Ω (conectado, enviando, sin red).
Alimentación	USB / 5V	5 V	Cargador de pared; power bank en el nodo portátil.

Terminal de tienda (ESP32-DevKitC)

Señal	Pin	Voltaje	Notas
I2C SDA / SCL	GPIO21 / GPIO22	3.3 V	Bus compartido: PN532 (0x24, modo I2C por sus interruptores) y LCD 1602 con PCF8574 (0x27).
Botones	GPIO32, 33, 25, 26	3.3 V	Anterior, siguiente, confirmar y cancelar; a GND con pull-up interno.
Buzzer activo	GPIO27	3.3 V lógica; buzzer a 5 V	Por 2N2222 con 1 kΩ en la base.
LED WS2812B	GPIO13	Datos 3.3 V; módulo a 5 V	Verde = registrado, rojo = advertencia de alérgeno.
LCD 1602	5V	5 V	Contraste ajustable en el módulo; pull-ups I2C a 3.3 V.

Terminal de recogida (ESP32-DevKitC)

Señal	Pin	Voltaje	Notas
I2C SDA / SCL	GPIO21 / GPIO22	3.3 V	PN532 y LCD 1602, igual que la terminal de tienda.
Teclado: filas	GPIO32, 33, 25, 26	3.3 V	Salidas que se activan en bajo una a la vez.
Teclado: columnas	GPIO27, 14, 4	3.3 V	Entradas con pull-up interno.
Buzzer activo	GPIO18	3.3 V lógica; buzzer a 5 V	Por 2N2222 con 1 kΩ en la base.
LED WS2812B	GPIO19	Datos 3.3 V; módulo a 5 V	Verde = recogida autorizada, rojo = rechazada.

Protocolos de comunicación

Enlace	Protocolo	Detalle	Justificación
Pulsera → nodo	Bluetooth Low Energy (NimBLE)	Anuncio no conectable; escaneo pasivo en el nodo (100 ms de intervalo, 50 ms de ventana).	Muy bajo consumo; un nodo recibe a todas las pulseras a la vez sin emparejar.
Nodo → pulsera*	Bluetooth Low Energy	Anuncio del nodo con un comando firmado; la pulsera escanea de forma periódica y breve.	Reutiliza la radio existente; no requiere conexión ni emparejamiento.
Pulsera o tarjeta → terminal*	NFC ISO/IEC 14443A	NTAG215 (pulsera) y MIFARE Classic 1K (tarjeta); se usa solo el UID.	Alcance de 2–4 cm: prueba que el alumno o el adulto está frente a la terminal.
Microcontrolador ↔ periféricos	I2C (100 kHz), GPIO, RMT	PN532 y LCD por I2C; teclado y botones por GPIO; WS2812 por RMT.	Pocos pines y controladores disponibles en ESP-IDF.
Nodo y terminales → red	WiFi IEEE 802.11 b/g/n, WPA2	Banda de 2.4 GHz, modo estación; coexistencia WiFi–BLE activa en el nodo.	Red ya existente en la escuela.
Dispositivos → API	HTTP/1.1 sobre TLS (HTTPS) + JSON	esp_http_client con el paquete de certificados de ESP-IDF. En la red local de desarrollo, HTTP.	Sin intermediarios (no se usa MQTT): la API valida y responde de inmediato cada envío.
Panel ↔ API	HTTPS + JSON	fetch del navegador; cookie de sesión HttpOnly y SameSite=Strict.	Un solo origen para el panel y la API; sin tokens en JavaScript.
API ↔ base de datos	TDS sobre TLS	Entity Framework Core con SQL Server o Azure SQL; reintentos automáticos.	Mismo motor en desarrollo y en la nube.
API → navegador*	Web Push (RFC 8030) con VAPID (RFC 8292)	Mensaje cifrado a través del servicio push del navegador.	Avisos con la app cerrada, sin depender de una app de mensajería.

Formato preliminar de mensajes

Anuncio BLE de la pulsera

Cabe en los 31 bytes de un anuncio BLE. El identificador de fabricante 0xFFFF está reservado para pruebas; el byte de versión permite cambiar el formato sin confundir a los nodos.

Bytes	Campo	Valor	Descripción
02 01 06	Flags	06	Descubrible general; sin Bluetooth clásico.
08 09 …	Nombre completo	«SB-0001»	Identificador de la pulsera (formato SB-XXXX).
05 FF FF FF	Datos de fabricante	FF FF	Identificador de fabricante de pruebas.
01	Versión	01	Versión del formato.
EE	Estado	bit0 a bit3	bit0 = puesta, bit1 = SOS, bit2 = batería baja, bit3 = confirmación de simulacro*.

Comando BLE del nodo a la pulsera*

Bytes	Campo	Descripción
FF FF 02	Fabricante y versión	Mismo identificador de pruebas; versión 02 = comando.
1 byte	Tipo	01 = simulacro, 02 = llamado de recogida, 03 = resultado de recogida.
2 bytes	Destino	Número de la pulsera (SB-0001 → 0x0001); 0xFFFF = todas (simulacro).
1 byte	Valor	Simulacro: 1 inicia, 0 termina. Resultado: 1 verde, 0 rojo.
4 bytes	Contador	Creciente; la pulsera descarta comandos repetidos.
4 bytes	Firma	HMAC-SHA256 truncado, con una clave de la escuela grabada en la pulsera.

Lectura del nodo (POST /api/lecturas)

```
POST /api/lecturas
Content-Type: application/json
X-Clave-Dispositivo: <clave del nodo>        (planeado, RNF-16)

{
  "nodo": "nodo-1",
  "pulsera": "SB-0001",
  "rssi": -67,
  "puesta": true,
  "sos": false,
  "bateriaBaja": false,
  "timestamp": "2026-10-12T15:04:05Z"     (opcional; si falta, la API usa su hora
UTC)
}
```

Respuesta 201 (lectura guardada):

```
{ "id": 1523, "nodo": "nodo-1", "zona": "Salón 3°A", "pulsera": "SB-0001",
  "alumno": "Ana López", "rssi": -67, "unidad": "dBm", "puesta": true,
  "sos": false, "bateriaBaja": false, "timestamp": "2026-10-12T15:04:05Z" }
```

Todos los errores de la API usan el mismo formato, con un mensaje en español y el código HTTP correspondiente (400 datos inválidos, 401 sin sesión o sin clave, 403 rol sin permiso, 404 no existe, 409 conflicto, 429 demasiados intentos, 500 error interno):

```
{ "error": "La pulsera 'SB-0099' no está registrada o está inactiva." }
```

Consulta del padre (GET /api/mis-hijos)

```
[ { "alumnoId": 7, "nombre": "Ana López", "parentesco": "Madre", "pulsera":
"SB-0001",
    "ultima": { "zona": "Patio", "rssi": -70, "puesta": true, "sos": false,
                "bateriaBaja": false, "timestamp": "2026-10-12T17:40:12Z" },
    "recorrido": [ { "zona": "Salón 3°A", "desde": "…T14:02Z", "hasta": "…T17:30Z"
},
```

```
                    { "zona": "Patio", "desde": "…T17:35Z", "hasta": "…T17:40Z" } ] }
]
```

Terminales*

```
POST /api/recogida/verificar   { "uidTarjeta": "04A2B91C", "pin": "4821" }
  → 200 { "recogidaId": "…", "alumnos": [ { "id": 7, "nombre": "Ana López" } ] }
POST /api/recogida/confirmar   { "recogidaId": "…", "uidPulsera": "04D3E2F1A07780" }
  → 200 { "resultado": "Autorizada", "alumno": "Ana López" }  |  { "resultado":
"NoCoincide" }
POST /api/tienda/consumos      { "uidPulsera": "04D3E2F1A07780", "productoId": 3,
"confirmar": false }
  → 409 { "advertencia": "Contiene cacahuate", "alergenos": ["Cacahuate"] }  |  201
registrado
```

Notificación push*

```
{ "tipo": "SOS", "titulo": "Ana activó el SOS", "cuerpo": "Patio, 11:40", "url":
"/#/hijos" }
```

Endpoints, eventos y operaciones principales

La API es REST sobre HTTP; no se usan tópicos ni un broker de mensajes. «Personal» significa Administrador o Maestro; «Dispositivo» significa un nodo o una terminal autenticado con su clave. Todos los endpoints de administración incluyen alta, consulta, edición, baja lógica (DELETE) y reactivación.

Método y ruta	Quién	Operación	Estado
POST /api/auth/login · /logout	Cualquiera	Iniciar y cerrar sesión (límite de 10 por minuto por IP en login).	Implementado
POST /api/auth/registro	Padre con invitación	Crear la cuenta con el código y aceptar el aviso de privacidad.	Implementado
GET /api/auth/yo	Con sesión	Datos y roles del usuario actual.	Implementado
POST /api/lecturas	Dispositivo (hoy abierto)	Recibir una lectura de un nodo.	Implementado (clave planeada)
GET /api/lecturas?limit=N	Personal	Últimas N lecturas de la escuela.	Implementado
GET /api/mis-hijos?desde=	Padre	Hijos del padre con su última detección y recorrido.	Implementado
/api/alumnos	Administrador	CRUD de alumnos con asignación de pulsera.	Implementado
/api/pulseras · PUT /{id}/alumno	Administrador	Inventario de pulseras y asignación.	Implementado
/api/tutores · POST /{id}/invitacion	Administrador	CRUD de tutores, vínculos con hijos e invitaciones.	Implementado
/api/zonas · /api/nodos	Administrador	CRUD de zonas y nodos.	Implementado
POST /api/nodos/{id}/clave · /api/terminales	Administrador	Registrar terminales y generar o revocar la clave de un dispositivo.	Planeado

Método y ruta	Quién	Operación	Estado
GET /api/comandos	Dispositivo (nodo)	Comandos BLE vigentes para su zona (consulta cada 3 s).	Planeado
POST /api/simulacro/iniciar · /terminar	Administrador	Activar el modo simulacro y ver las confirmaciones.	Planeado
POST /api/recogida/llamar	Personal	Llamar a un alumno a la salida (pulsera en azul).	Planeado
POST /api/recogida/verificar · /confirmar	Dispositivo (terminal)	Validar tarjeta y PIN, y emparejar con la pulsera.	Planeado
GET /api/tienda/productos · POST /consumos	Dispositivo (terminal)	Catálogo y registro de consumo con alérgenos.	Planeado
GET /api/alertas · POST /{id}/atender	Personal	Alertas abiertas y su atención.	Planeado
GET · PUT /api/asistencia	Maestro	Pase de lista del grupo y corrección.	Planeado
POST /api/push/suscripciones	Con sesión	Registrar el navegador para notificaciones.	Planeado

Eventos internos del backend*

Las reglas que dependen del tiempo o de varias lecturas se ejecutan dentro de la API, sin componentes adicionales:

- **Lectura guardada:** el motor de alertas revisa SOS y pulsera retirada y, si corresponde, crea una alerta.
- **Tarea periódica (cada 60 s):** un servicio en segundo plano detecta alumnos sin señal en horario escolar y salidas del perímetro sin recogida en ±10 min.
- **Alerta o recogida creada:** el notificador envía la notificación push a los padres y al personal suscritos.

Modelo de autenticación

Usuarios

- ASP.NET Core Identity guarda las cuentas con contraseña cifrada mediante hash PBKDF2 (mínimo 8 caracteres con mayúscula, minúscula y número) y bloquea la cuenta 15 minutos tras 5 intentos fallidos.
- La sesión es una cookie cifrada «SafeBand.Sesion», HttpOnly y SameSite=Strict, con vigencia de 7 días renovable. JavaScript no puede leerla, lo que protege contra robo por XSS.
- Tres roles con políticas: SoloAdministrador, Personal (Administrador o Maestro) y SoloPadre. Además del rol, cada consulta del padre se filtra por los alumnos vinculados a su tutor.
- Las cuentas de padre solo se crean con un código de invitación de un solo uso (vigencia de 7 días, guardado como hash); no hay registro abierto. El administrador inicial se crea al arrancar desde la configuración secreta.
- El encargado de la tienda y el personal de la entrada no necesitan cuenta: operan terminales, que se autentican como dispositivos.

Dispositivos*

- Cada nodo y cada terminal tiene una clave aleatoria de 32 bytes que el administrador genera desde el panel; se muestra una sola vez y en la base de datos solo se guarda su hash SHA-256.
- El dispositivo la envía en el encabezado X-Clave-Dispositivo por HTTPS. La API rechaza con 401 las peticiones sin clave, con clave revocada o con la clave de otro dispositivo (RNF-16).
- Si un dispositivo se pierde, se revoca su clave y se genera una nueva, sin afectar a los demás.

Pulseras

La pulsera no se autentica ante la API: solo anuncia su identificador, que podría imitarse con otro dispositivo BLE. En el prototipo se acepta este riesgo, porque la pulsera solo informa ubicación y la autorización de recogida nunca depende de BLE: exige tarjeta, PIN y el tap físico de la pulsera en la terminal. Como mejora futura se documenta un anuncio con contador firmado. En sentido contrario, los comandos que la pulsera recibe sí van firmados, para que nadie pueda cambiar su color o activar un simulacro falso.

Protección de credenciales y variables de entorno

Regla general: ningún secreto se escribe en el código ni se sube al repositorio de GitHub. appsettings.json solo contiene valores no sensibles.

Secreto	Desarrollo	Producción	Protección
Cadena de conexión a la base de datos	appsettings de desarrollo con LocalDB (autenticación de Windows, sin contraseña)	Cadena de conexión de App Service	Cifrada en reposo por Azure; conexión TLS a Azure SQL.
Usuario y contraseña del administrador inicial	user-secrets de .NET (fuera del proyecto)	Configuración de App Service	Solo se usan al sembrar; se cambia la contraseña tras el primer inicio.
Claves de Data Protection (cifran la cookie)	Perfil del usuario de Windows	Almacenamiento persistente de App Service	Gestionadas por ASP.NET Core; sobreviven a reinicios.
Clave privada VAPID (push)*	user-secrets	Configuración de App Service	Nunca se envía al navegador; solo la clave pública.
WiFi (SSID y contraseña) y URL de la API	menuconfig → sdkconfig	sdkconfig de cada placa	sdkconfig está en .gitignore; solo sdkconfig.defaults se sube.
Clave de dispositivo*	menuconfig → sdkconfig	Partición NVS de la placa	En la BD solo el hash; revocable desde el panel.
Clave de firma de comandos*	menuconfig	NVS de pulseras y API	Cifrado de flash y NVS de ESP-IDF previsto para producción.
Contraseñas, códigos y PIN de usuarios	—	—	Solo hash (PBKDF2); nunca en texto legible.

Comportamiento sin Internet y después de un reinicio

Componente	Sin Internet o sin red	Después de un reinicio
Pulsera	No depende de Internet: sigue anunciando y conserva el SOS, que el nodo entregará cuando tenga red.	En menos de 2 s lee el broche y vuelve a anunciar. Un SOS en curso se pierde y debe repetirse; el simulacro se restablece porque el nodo lo sigue anunciando*.
Nodo	Sigue escaneando y reintenta el WiFi solo. Hoy descarta las lecturas sin red; planeado: búfer circular en RAM de 30 min con hora UTC (RNF-09).	Lee su configuración de la flash, se reconecta y continúa sin intervención. Planeado: sincronizar la hora por SNTP y configurar el watchdog de tareas para reiniciar la placa si una tarea se bloquea*.
Terminal de tienda*	No registra consumos ni valida alérgenos; la LCD indica «Sin conexión» para que el encargado verifique manualmente.	Se reconecta y descarga de nuevo el catálogo.
Terminal de recogida*	No autoriza ninguna recogida; la LCD indica «Sin conexión» y el personal aplica el procedimiento manual.	Se reconecta; las recogidas a medio verificar caducan en la API.
API	Si la escuela pierde Internet deja de recibir lecturas; al volver, el panel muestra el hueco. Si la BD falla, EF Core reintenta y después responde 500 sin detalles.	Aplica las migraciones pendientes y el sembrado (idempotente) al arrancar. Las sesiones siguen válidas porque las claves de Data Protection son persistentes.
Panel web	Muestra «Sin conexión», conserva los últimos datos y reintenta cada 5 o 10 s. Con PWA*, abre desde la caché con los últimos datos.	Si la cookie sigue vigente, entra directo; si venció, pide iniciar sesión.
Notificaciones push*	El servicio push guarda la notificación hasta que el celular se conecte.	No aplica.

Tecnologías y versiones principales

Capa	Tecnología	Versión	Uso
Microcontrolador de la pulsera	Seeed XIAO ESP32-C3 (RISC-V, BLE 5)	—	Pulsera.
Microcontrolador de nodo y terminales	ESP32-DevKitC (ESP32-WROOM-32, WiFi + BLE 4.2)	—	Nodos y terminales.
Firmware	ESP-IDF (C) con NimBLE, esp_wifi, esp_http_client y esp-tls	6.0.2	Todos los dispositivos.
Lectores y pantallas	PN532 (NFC), LCD 1602 con PCF8574, WS2812B	—	Terminales y pulsera.
Backend	.NET y ASP.NET Core (Minimal API)	10	API REST.
Autenticación	ASP.NET Core Identity (EF Core)	10.0.12	Usuarios, roles y contraseñas.
Acceso a datos	Entity Framework Core (SQL Server)	10.0.12	Modelo, migraciones y consultas.
Documentación de la API	Microsoft.AspNetCore.OpenApi y Swagger UI	10.0.12 / 10.2.3	Prueba de endpoints.
Base de datos	SQL Server LocalDB / Azure SQL Database	—	Desarrollo / producción.

Capa	Tecnología	Versión	Uso
Frontend	HTML5, CSS3 y JavaScript (módulos ES), sin framework	—	Panel web.
PWA y push*	Service Worker, Web App Manifest, Web Push API (VAPID)	—	Instalación y notificaciones.
Nube	Azure App Service (plan F1) y Azure SQL, región Mexico Central	—	Despliegue (RNF-15).
Herramientas	Visual Studio, VS Code con la extensión ESP-IDF, Git y GitHub	—	Desarrollo y control de versiones.

Las versiones de bibliotecas secundarias (controladores del PN532 y de la LCD, biblioteca de Web Push) se fijarán al incorporarlas y quedarán registradas en el archivo de proyecto correspondiente.

Matriz de requisitos y componentes

Cada requisito tiene al menos un componente responsable. Se agrega RF-22, propuesto en esta etapa por la función de llave de recogida, para incorporarlo al Documento de Requisitos.

ID	Requisito	Pulsera	Nodo	T. tienda	T. recogida	API	BD	Frontend	Azure	Módulo responsable
RF-01	Anuncio BLE con estado	X								Firmware pulsera: anuncio
RF-02	Botón SOS de 2 s	X								Firmware pulsera: entradas
RF-03	Pulsera retirada	X	X							Firmware pulsera: entradas
RF-04	Batería baja	X								Firmware pulsera: entradas (ADC)
RF-05	Escaneo y promedio		X							Firmware nodo: acumulador
RF-06	SOS inmediato		X			X				Firmware nodo: envío HTTP
RF-07	Recepción de lecturas					X	X			LecturasEndpoints
RF-08	Historial de lecturas					X	X			LecturasEndpoints
RF-09	Inicio de sesión por rol					X		X		AuthEndpoints, app.js
RF-10	Gestión de alumnos					X	X	X		AlumnosEndpoints, vista alumnos
RF-11	Inventario de pulseras					X	X	X		PulserasEndpoints, vista pulseras
RF-12	Zonas y nodos					X	X	X		ZonasNodosEndpoints, vista zonas
RF-13	Tutores y vínculos					X	X	X		TutoresEndpoints, vista tutores
RF-14	Invitación y registro					X	X	X		TutoresEndpoints, AuthEndpoints
RF-15	Mis hijos					X		X		MisHijosEndpoints, vista hijos
RF-16	Vista En vivo					X		X		Vista vivo
RF-17	Alertas					X	X	X		Motor de alertas*, notificador push*

ID	Requisito	Pulsera	Nodo	T. tienda	T. recogida	API	BD	Frontend	Azure	Módulo responsable
RF-18	Pase de lista					X	X	X		Asistencia*, tarea periódica*
RF-19	Terminal de tienda			X		X	X	X		Firmware tienda*, TiendaEndpoints*
RF-20	Terminal de recogida				X	X	X			Firmware recogida*, RecogidaEndpoints*, cruce*
RF-21	Trilateración		X			X		X		Servicio de ubicación*, vista plano*
RF-22*	Llave de recogida en la pulsera	X	X		X	X				Comandos BLE*, RecogidaEndpoints*
RNF-01	Contraseñas seguras					X	X			Identity (Program.cs)
RNF-02	Acceso por rol					X				Políticas de autorización
RNF-03	Sesión segura					X			X	Cookie de Identity, HTTPS
RNF-04	Fuerza bruta					X				Limitador «auth»
RNF-05	Privacidad (LFPDPPP)					X	X			Esquema mínimo, invitaciones con hash
RNF-06	Latencia extremo a extremo		X			X		X		Ventana del nodo, consultas del panel
RNF-07	Volumen y rendimiento		X				X			Agregación por ventana, índices
RNF-08	Reconexión del nodo		X							Firmware nodo: WiFi
RNF-09	Guardar y reenviar		X			X				Búfer sin red*
RNF-10	Recuperación del panel							X		api.js, vistas
RNF-11	Integridad del historial					X	X			AppDbContext, bajas lógicas
RNF-12	Usabilidad y accesibilidad							X		app.css, ui.js
RNF-13	Autonomía de la pulsera	X								Firmware pulsera: intervalos BLE
RNF-14	Alcance BLE	X	X							Umbral RSSI por nodo
RNF-15	Despliegue en la nube					X	X		X	App Service, Azure SQL

ID	Requisito	Pulsera	Nodo	T. tienda	T. recogida	API	BD	Frontend	Azure	Módulo responsable
RNF-16	Autenticación de dispositivos		X	X	X	X	X			Autenticación de dispositivos*

Cambios respecto a documentos anteriores y control de cambios

Al definir la arquitectura se incorporaron las siguientes decisiones, que actualizan lo descrito en el Documento de Requisitos y en los Modelos del comportamiento:

- **Llave de recogida (RF-22, deseable):** el adulto se identifica con tarjeta y PIN y, además, el alumno acerca su pulsera al mismo lector NFC de la terminal; la API comprueba que el adulto pueda recoger a ese alumno. La pulsera muestra azul al ser llamada y verde o rojo según el resultado, mediante comandos BLE firmados.
- **Canal BLE del nodo a la pulsera:** un solo mecanismo para el modo simulacro y para la llave de recogida, con firma y contador para evitar comandos falsos o repetidos.
- **Autenticación de dispositivos (RNF-16):** nodos y terminales usan una clave por dispositivo; el personal de tienda y de entrada no requiere cuenta en el panel.
- **Sin broker de mensajes:** los dispositivos hablan directo con la API por HTTPS y los nodos consultan sus comandos de forma periódica; no se usan MQTT ni ESP-NOW (queda como trabajo futuro).

A partir de esta entrega, la arquitectura queda congelada. Todo cambio en componentes, protocolos, formatos de mensaje, endpoints principales o modelo de autenticación se registrará en la bitácora del proyecto (MANUAL.md, en el repositorio) con la fecha, el cambio, el motivo, los requisitos afectados y los documentos que deben actualizarse. Los ajustes de parámetros preliminares (pines, tiempos y tamaños de búfer) se anotan en la bitácora, pero no requieren justificación de cambio de arquitectura.